

C++ COMPETITIVE PROGRAMMING + DATA STRUCTURES + ALGORITHMS MASTER TEXTBOOK

From Absolute Beginner to Advanced Competitive
Programmer

**MASTER ROADMAP · TABLE OF CONTENTS · MASTER
CHECKLIST**

This volume is the map for the whole course. It lists every Part, the 24-phase learning path, how the book fits together, and a master checklist you can tick off as you progress. Keep this file open in a tab while you work through the other Parts.

How This Book Works

This is not a university-syllabus summary. It is a long-form course that takes you from "I barely know C++" to "I can read an unfamiliar problem, figure out the constraints, choose the right data structure and algorithm, prove why it works, code it cleanly in C++, and debug it under time pressure." That last skill — turning an unfamiliar problem into a solved one — is the actual goal. Syntax is just the tool.

The book is delivered as a series of PDFs, one per Part (28 Parts total, matching the structure below). Each Part is self-contained: it can be read on its own, but Parts build on each other in order, so work through them in sequence the first time.

Every chapter inside a Part follows the same rhythm, so you always know what to expect:

- 1 One simple sentence explaining the idea.
- 2 Intuition — why it works, in plain language.
- 3 A small worked example.
- 4 The C++ code.
- 5 A line-by-line explanation of the code.
- 6 A dry run (tracing the code by hand on a real input).
- 7 Time and space complexity.
- 8 Common mistakes beginners make with this exact topic.
- 9 Practice problems, organized from Level 0 (absolute beginner) to Level 6 (hard).

▢ Reading tip

Do not skip the dry runs. Reading code and tracing code by hand are different skills — competitive programmers trace code in their heads constantly, and it is a trained habit, not a talent.

The 24-Phase Learning Path

This is the order your brain should learn things in, independent of how the PDFs are split:

Phase	Focus
Phase 1	C++ Fundamentals
Phase 2	Problem-Solving Fundamentals
Phase 3	Arrays and Strings
Phase 4	STL
Phase 5	Complexity
Phase 6	Searching and Sorting
Phase 7	Hashing
Phase 8	Linked Lists
Phase 9	Stacks and Queues
Phase 10	Recursion and Backtracking
Phase 11	Trees
Phase 12	Heaps
Phase 13	Graphs
Phase 14	Greedy
Phase 15	Dynamic Programming
Phase 16	Bit Manipulation
Phase 17	Number Theory
Phase 18	Advanced Strings
Phase 19	Advanced Data Structures
Phase 20	Advanced Algorithms
Phase 21	Competitive Programming
Phase 22	Codeforces
Phase 23	LeetCode
Phase 24	Advanced Problem Solving

Full Book Structure

Each row below is one delivered PDF. Tick the checkbox in the last section as you finish a Part.

Part I — C++ Fundamentals

Programming basics, syntax, variables, data types, I/O, operators, conditions, loops, functions, pointers & references.

Part II — Problem-Solving Fundamentals

How to think before coding, brute force → optimization, complexity intuition, reading constraints.

Part III — Arrays and Strings

Deep array and string mastery: traversal, manipulation, 2D arrays, string algorithms basics, vectors.

Part IV — C++ STL

vector, pair, map, set, stack, queue, priority_queue, unordered_*, algorithms, iterators, competitive patterns.

Part V — Complexity

Big-O from zero, analyzing loops/recursion/STL/sorting/graphs/DP, best/average/worst case.

Part VI — Searching and Sorting

Linear/binary search (deep), all classic sorts, binary search on the answer.

Part VII — Hashing

Hash tables, collisions, unordered_map/set, frequency techniques.

Part VIII — Linked Lists

Singly/doubly/circular lists, reversal, cycle detection, merging.

Part IX — Stacks, Queues and Deques

Implementations, monotonic stack, circular queue, STL versions.

Part X — Recursion and Backtracking

Call stack, recursion patterns, permutations/subsets, N-Queens, pruning.

Part XI — Trees and BST

Traversals, height/diameter, BST operations, tree problem patterns.

Part XII — Heaps

Min/max heap, heapify, heap sort, priority_queue applications.

Part XIII — Graphs

Representations, BFS/DFS, shortest paths, MST, DSU, SCC, bridges/articulation points.

Part XIV — Greedy Algorithms

Exchange argument, interval scheduling, fractional knapsack, when greedy fails.

Part XV — Dynamic Programming

State/transition thinking, 1D/2D DP, knapsack family, string DP, tree DP, bitmask DP, digit DP.

Part XVI — Mathematics and Number Theory

Primes, sieve, GCD/LCM, modular arithmetic, combinatorics, matrix exponentiation.

Part XVII — Advanced String Algorithms

Hashing, KMP, Z-algorithm, Trie, Aho-Corasick introduction.

Part XVIII — Advanced Data Structures

Fenwick Tree, Segment Tree (+ lazy propagation), Sparse Table, Binary Lifting/LCA.

Part XIX — Advanced Algorithms

Divide & conquer, meet in the middle, coordinate compression, sweep line, small-to-large.

Part XX — C++ OOP / University Module

Classes, constructors, operator overloading, inheritance, polymorphism, templates.

Part XXI — Exceptions, Files and Templates

try/catch, file streams, function & class templates.

Part XXII — Competitive Programming Patterns

Pattern recognition guide, constraints → algorithm selection.

Part XXIII — Codeforces Roadmap

Rating system, contest strategy, rating-by-rating skill map (800 → 2000+).

Part XXIV — LeetCode Roadmap

Easy → Medium → Hard progression, pattern-based problem sets.

Part XXV — Interview Preparation

C++/STL/DSA conceptual questions with reasoned answers.

Part XXVI — Mock Contests

Contests by rating band plus LeetCode-style sets, solutions separated.

Part XXVII — Master Revision & Cheat Sheets

One-page cheat sheets for every major topic.

Part XXVIII — Capstone

Daily/weekly practice structure, long-term roadmap, final master checklist.

Two Habits That Matter More Than Any Single Topic

1. Read constraints before choosing an algorithm

Competitive problems tell you the intended complexity through their constraints. This is a rough guide, not a law — always sanity-check against the specific problem:

Constraint on n	Likely intended complexity
$n \leq 10$	$O(n!)$ or exhaustive search may be intended
$n \leq 20$	$O(2^n)$ — bitmask / subset enumeration
$n \leq 40$	$O(2^{(n/2)})$ — meet in the middle
$n \leq 100$	$O(n^3)$ may work
$n \leq 1,000$	$O(n^2)$ may work
$n \leq 100,000$	$O(n \log n)$ usually expected
$n \leq 1,000,000$	$O(n)$ or $O(n \log n)$

2. Separate "brute force" from "optimal" as two distinct steps

Almost every hard problem is solved by first finding a slow, obviously-correct brute force, then asking: what is the bottleneck, and what observation removes it? This book repeats that cycle — Naive → Why it's slow → Observation → Improvement → Optimal — for the majority of non-trivial problems, because that cycle is competitive problem solving.

Master Checklist

Tick these off as you complete each Part. This list mirrors the 24-phase path, not the PDF order, so a few boxes will be completed gradually across more than one Part.

Foundations

- ☐ C++ basics (syntax, variables, data types)
- ☐ Input / output & fast I/O
- ☐ Operators
- ☐ Conditions
- ☐ Loops
- ☐ Functions & recursion basics
- ☐ Pointers & references

Core Data Handling

- ☐ Arrays (1D & 2D)
- ☐ Strings
- ☐ STL containers
- ☐ STL algorithms
- ☐ Complexity analysis

Searching, Sorting, Hashing

- ☐ Linear & binary search
- ☐ Binary search on the answer
- ☐ Classic sorting algorithms
- ☐ Hash tables / unordered_map / unordered_set

Linear Structures

- ☐ Linked lists
- ☐ Stacks (incl. monotonic stack)
- ☐ Queues & dequeues
- ☐ Priority queues / heaps

Recursion Family

- ☐ Recursion patterns
- ☐ Backtracking
- ☐ Divide and conquer

Trees & Graphs

- ☐ Binary trees & traversals
- ☐ Binary search trees
- ☐ Graph representations
- ☐ BFS / DFS
- ☐ Shortest paths (Dijkstra, Bellman-Ford, Floyd-Warshall)
- ☐ MST (Kruskal, Prim)
- ☐ DSU
- ☐ Topological sort
- ☐ SCC / bridges / articulation points

Algorithm Design

- ☐ Greedy algorithms
- ☐ Dynamic programming
(1D/2D/knapsack/string/tree/bitmask/digit)
- ☐ Bit manipulation

Mathematics

- ☐ Number theory basics
- ☐ Modular arithmetic
- ☐ Combinatorics
- ☐ Matrix exponentiation

Advanced Topics

- ☐ String hashing, KMP, Z-algorithm, Trie
- ☐ Fenwick Tree
- ☐ Segment Tree (+ lazy propagation)
- ☐ Sparse Table

- ☐ Binary lifting / LCA
- ☐ Meet in the middle, coordinate compression, sweep line

University Module

- ☐ OOP (classes, inheritance, polymorphism)
- ☐ Operator overloading
- ☐ Templates
- ☐ Exceptions & file I/O

Applied Practice

- ☐ Pattern-recognition drills
- ☐ Codeforces practice by rating band
- ☐ LeetCode practice by pattern
- ☐ Mock contests
- ☐ Interview-style Q&A;

What's Next

The next file is Part I — C++ Fundamentals. It starts from zero: what programming even is, through variables, data types, I/O, operators, conditions, loops, functions, and a first look at pointers and references — always tied back to why each piece matters for competitive programming, not just university syntax.

□ A note on pace

Do not rush Part I even if you already know some C++. The goal is not "I have seen this syntax before" — it is "I understand this well enough to use it instantly, correctly, and fast, under contest pressure."